Detalles de la tarea de esta unidad.

Enunciado.

A partir de los recursos de la unidad 6, en esta tarea debes realizar las pruebas unitarias de tu aplicación de reservas.

Al ser una aplicación cliente que recoge datos, debemos centrarnos en verificar que los **eventos** se gestionan correctamente, es decir, que se comporta como esperamos:

- Las opciones de navegación funcionan: Al pulsar un botón o una opción de menú se muestra el formulario de reserva, y se cierra al pulsar en el botón correspondiente.
- Los campos están inicialmente vacíos y las validaciones funcionan.
- Al seleccionar una opción concreta, se habilitan ciertos campos, quedando deshabilitados en caso contrario, etc.

Para conseguir realizar las pruebas anteriores, hay que tener en cuenta las siguientes observaciones:

- La simulación de los eventos la vamos a gestionar en lo que se denomina hilo Event Dispatching Thread (EDT); sin embargo, del test se va a encargar el hilo principal (Main Thread). Por el motivo explicado aquí, y para evitar problemas, la clase de tests y cada bloque a probar debemos encerrarlo dentro de SwingUtilities.invokeLater(new Runnable() { ..... }};

```java
public class ReservaTest {

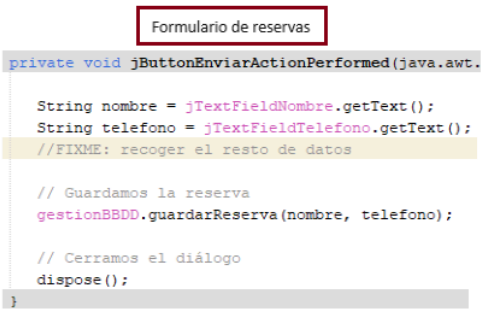
    private static Reservas reserva;

    public ReservaTest() {
    }

    @BeforeClass
    public static void setUpClass() {
        // Leer https://www.thegeekyway.com/edt_java/
        SwingUtilities.invokeLater(new Runnable() {
        public void run() {
            reserva = new Reservas(new InitialJFrame(), true);
        }
    });
    }

    // Prueba para validar los campos de nombre y telefono
    @Test
    public void testFormularioValidation() {
        // Metemos todos los componentes GUI en un bloque
        SwingUtilities.invokeLater(new Runnable() {
            public void run() {
```

- Ahora vamos a crear una estructura más parecida a una aplicación completa.
- Para ello, tu aplicación de reservas va a tener, además de la parte cliente donde se recogen datos, una parte servidor donde se debería procesar y almacenar la información. ¿Cómo vamos a comunicar el formulario con esta nueva parte? Pues por medio del botón "Reservar", que al ser pulsado recopilará todos los datos y se los enviará al servidor.
- Esta funcionalidad no es objetivo de este curso, pero sí podemos hacer que el botón Reservar envíe todos los datos de la reserva a una nueva clase llamada gestionBBDD, que dejaremos vacía (aunque debería ser una clase muy compleja que conectara a una base de datos, etc.).
- Ahora que ya tenemos una estructura más "real", son evidentes las dependencias entre el formulario que estamos probando con otras clases externas. Si queremos terminar de hacer los tests unitarios y probar la funcionalidad del botón Reservar de forma **aislada**, debemos recurrir al **Framework Mockito**.
- Lo recomendable, dada la esencia de las ***pruebas unitarias***, es centrarse en comprobar que Reservar hace lo que debe, es decir, recoge los datos del formulario y se los envía a otra clase diferente a la clase *Formulario* que estamos probando. Para hacer esto, hay que "mockear" la clase GestionBBDD:





  - NOTA: En este caso no sería muy costoso crear un objeto de tipo GestionBBDD, pero imagínate que fuera un caso real donde tienes que montar la conexión a la BBDD, procesar la respuesta, etc. Esto nos lo ahorramos con Mockito.

- Investiga cómo se puede revisar el porcentaje de cobertura de código testeado (puedes descargar el plugin de JaCoCo desde https://github.com/DaveBrad/dbrad-jacocoverage), y aplícalo a tus pruebas.

Criterios de puntuación. Total 10 puntos.

1. Realizados tests unitarios para las principales partes de la aplicación. 3 puntos.
2. Realizado test empleando mockito para aislar a la clase y salvar las dependencias con otras clases. 3 puntos.
3. Incorporado plugin de cobertura de pruebas en el código y revisado porcentaje para las clases de la aplicación de reservas. 2 puntos.
4. Documento explicativo de todo el proceso (en formato .pdf e incorporando pantallazos). 2 puntos.

Indicaciones de entrega.

Una vez realizada la tarea debes entregar:

- Un directorio llamado **leeme** donde incluyas el documento detallando el desarrollo que has seguido para la realización de la tarea (incluye pantallazos).
- Un directorio llamado **proyecto** con el proyecto completo, incluyendo las clases de tests, las cuales deben pasar, es decir, deben tener éxito.

Realiza un fichero comprimido con las siguientes pautas:

apellido1_apellido2_nombre_SIGxx_Tarea

Asegúrate que el nombre no contenga la letra ñ, tildes ni caracteres especiales extraños. Así por ejemplo la alumna **Begoña Sánchez Mañas para la octava unidad del MP de DI**, debería nombrar esta tarea como...

sanchez_manas_begona_DI06_Tarea

Agregar entrega

Estado de la entrega

Número del intento	Este es el intento 1 (2 intentos permitidos).
Estado de la entrega	Todavía no se han realizado envíos
Estado de la calificación	Sin calificar